

FTD Master

AI-Powered ESG Analysis

Session 4 – Evaluation, Advanced RAG, Reranking & MCP

Theory & Hands-on Application

Rami El Mnebhi

May 2026

EVALUATION

Why Evaluate RAG Pipelines?

Building a RAG system is easy. Building a trustworthy RAG system is hard.

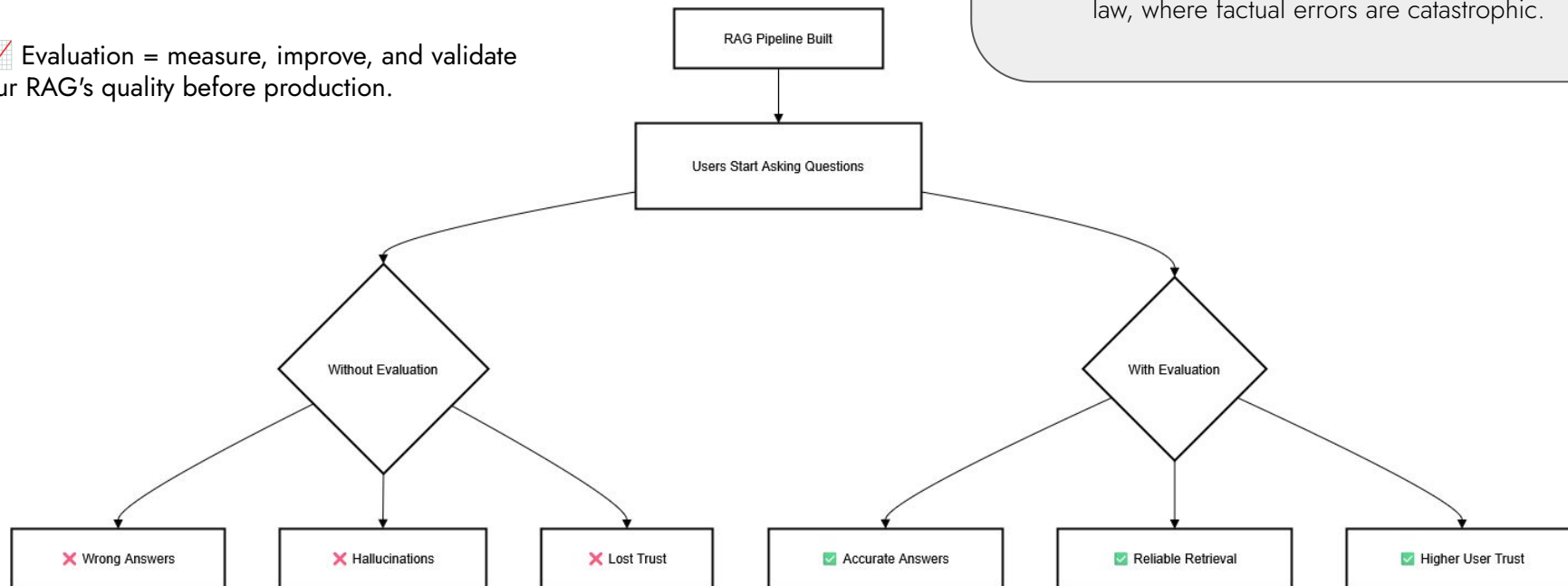
Without proper evaluation:

- Answers may hallucinate
- Retrieved context may be irrelevant
- Users lose trust in the assistant



Evaluation = measure, improve, and validate our RAG's quality before production.

- Classic "accuracy" metrics don't apply directly to RAG.
- A RAG pipeline needs faithfulness (does it quote the right context?) and relevance (does it answer well?).
- Evaluation is critical in fields like banking, ESG, law, where factual errors are catastrophic.



How to Evaluate a RAG Pipeline?

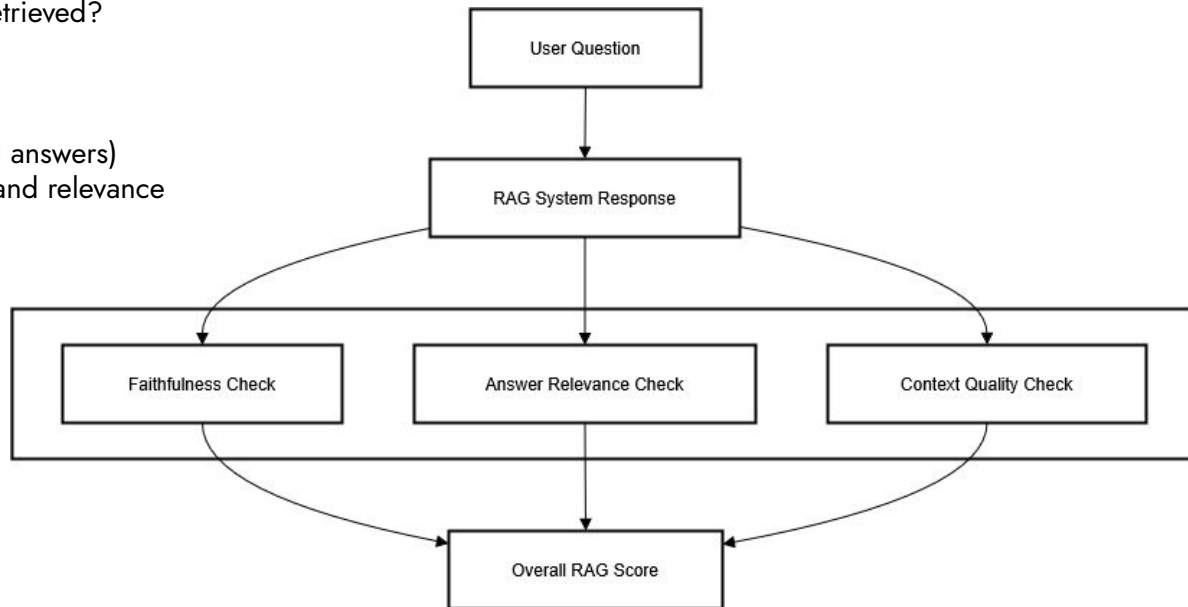
Traditional machine learning metrics (accuracy, precision) are not enough for RAG.

RAG-specific evaluation measures:

- **Faithfulness:** Is the answer supported by retrieved documents?
- **Answer Relevance:** Does the answer actually address the question?
- **Context Precision:** Are retrieved chunks tight and relevant?
- **Context Recall:** Are all necessary pieces retrieved?

- 🧠 To evaluate, we need:
- Ground-truth datasets (questions + expected answers)
 - LLM-as-a-Judge to assess subtle correctness and relevance

- 
- RAG is not a **classification task** — evaluation is softer, more subjective.
 - Human-like evaluation (via LLMs) is becoming standard: it captures nuance humans expect.
 - RAGAS automates this workflow



Introducing RAGAS: The RAG Evaluation Framework

RAGAS = Retrieval-Augmented Generation Assessment.

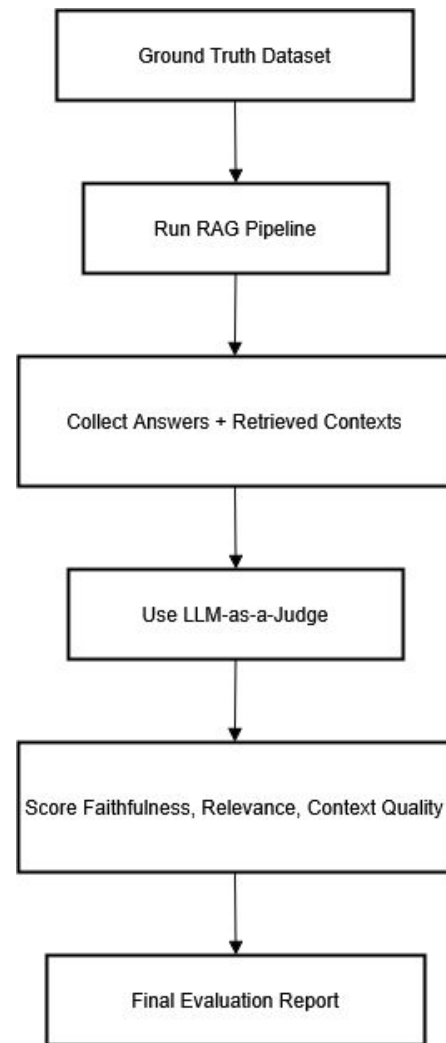
It provides:

-  Standardized Metrics for RAG quality
-  LLM-as-a-Judge to score faithfulness, relevance, context
-  Easy API to plug into any RAG pipeline
-  Automated reports to visualize weaknesses

RAGAS = Fast, objective, reproducible evaluation for RAG systems.

- 
- 
- Without tools like RAGAS, evaluation is slow, manual, subjective.
 - RAGAS leverages powerful LLMs (e.g., Albert API / GPT-4o) to simulate human judgment at scale.
 - RAGAS makes quality quantifiable — a necessity in banking, ESG, legal domains.

Link to doc ⇒ <https://docs.ragas.io/en/stable/>



Core Metrics in RAG Evaluation

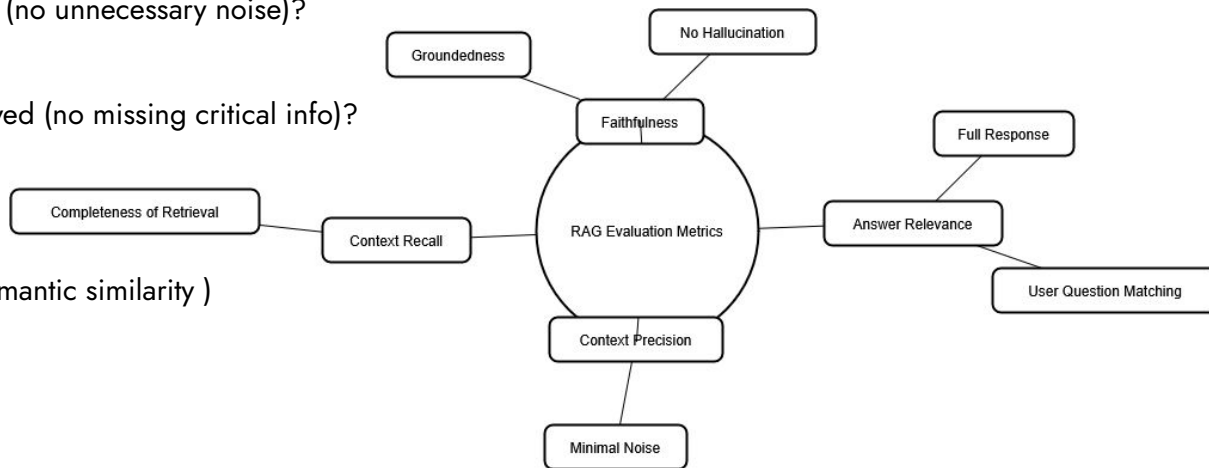
RAGAS uses four key metrics to evaluate your system:

-  **Faithfulness:**
Is the generated answer fully grounded in the retrieved documents?
(Avoids hallucinations)
-  **Answer Relevance:**
Does the answer fully address the user's original question?
-  **Context Precision:**
Are the retrieved chunks precise and minimal (no unnecessary noise)?
-  **Context Recall:**
Are all necessary pieces of information retrieved (no missing critical info)?

 Bonus Metrics:

- F1 Score (balance of precision & recall)
- Answer Correctness (strict factual match / semantic similarity)

- Faithfulness is critical in sensitive fields (finance, ESG, healthcare).
- Answer Relevance matters because LLMs sometimes answer plausibly but not accurately.
- Context Precision/Recall directly measure retrieval system quality (not just LLM "guesswork").
- These metrics together paint a full picture: no single metric is enough alone.



How LLM-as-a-Judge Works

LLMs are asked to simulate human judgment on RAG outputs.

🔍 For each (Question, Retrieved Context, Answer):

- Faithfulness Prompt:

"Is this answer factually supported by the retrieved context?"

- Relevance Prompt:

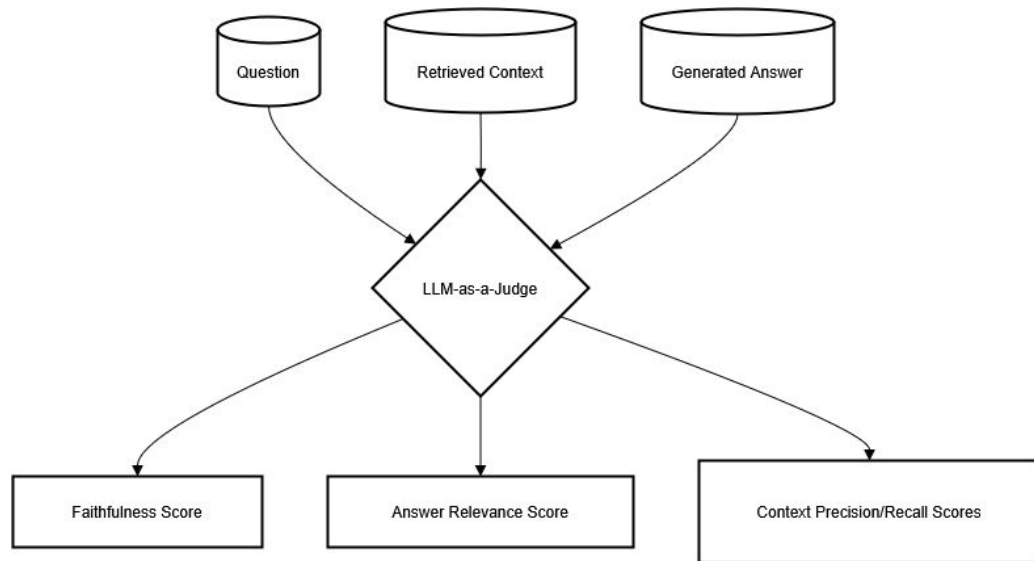
"Does the answer fully address the question?"

- Precision/Recall Prompts:

"Does the retrieved context contain all necessary information, and only what is needed?"

📊 The LLM scores responses on a 0–1 scale.)

- LLM-as-a-Judge is faster, cheaper, and often more consistent than manual human annotation.
- It's not perfect — garbage inputs still cause garbage judgments — but it scales.
- RAGAS provides standard judge prompts optimized for clarity and reproducibility.



Putting It All Together: RAGAS Evaluation Pipeline

1.  Prepare Evaluation Dataset
 - Questions + Ground-truth Answers
1.  Run Your RAG Pipeline
 - For each question, generate an answer + store retrieved contexts
1.  Use RAGAS to Evaluate
 - LLM-as-a-Judge scores Faithfulness, Relevance, Context Quality
1.  Analyze Results
 - Identify weaknesses (bad retrieval, hallucination, incomplete answers)

 **Bonus:** Automate this regularly to catch regressions when models or corpuses evolve!

- Systematic evaluation allows iteration and trust-building.
- RAGAS can be fine-tuned: you can customize evaluation prompts if needed.

ADVANCED RAG

Reranking: Getting the Right Chunks to the Top

The problem: vector similarity finds semantically close chunks — but close is not the same as relevant. Top-K retrieval has high recall but low precision.



What is Reranking?

- After initial retrieval (top-K by cosine), a reranker scores each chunk against the query with a more powerful cross-encoder model
- **Result:** chunks are reordered so the most relevant ones appear first — the LLM sees better context



Two-Stage Retrieval Pipeline

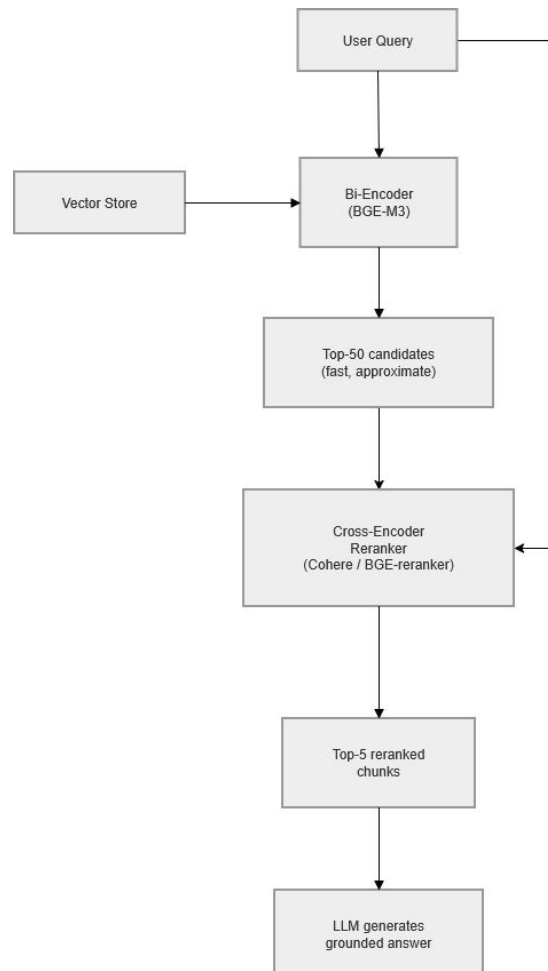
- **Stage 1 — Retrieve:** fast bi-encoder (e.g. BGE-M3) → top-50 candidates.
- **Stage 2 — Rerank:** slower cross-encoder (e.g. Cohere Rerank, BGE-reranker) → top-5 final



Compatible with HyDE and MQR — these are orthogonal improvements that stack



Bi-encoder vs Cross-encoder: bi-encoders embed query and doc independently (fast, scalable). Cross-encoders attend to both simultaneously (slower, much more accurate).



Contextual Chunking: Fixing the Lost-in-Isolation Problem

The problem: naive chunking strips context. A chunk saying "It increased by 12%" is meaningless without knowing what "it" refers to.



Contextual Chunking (Anthropic, 2024)

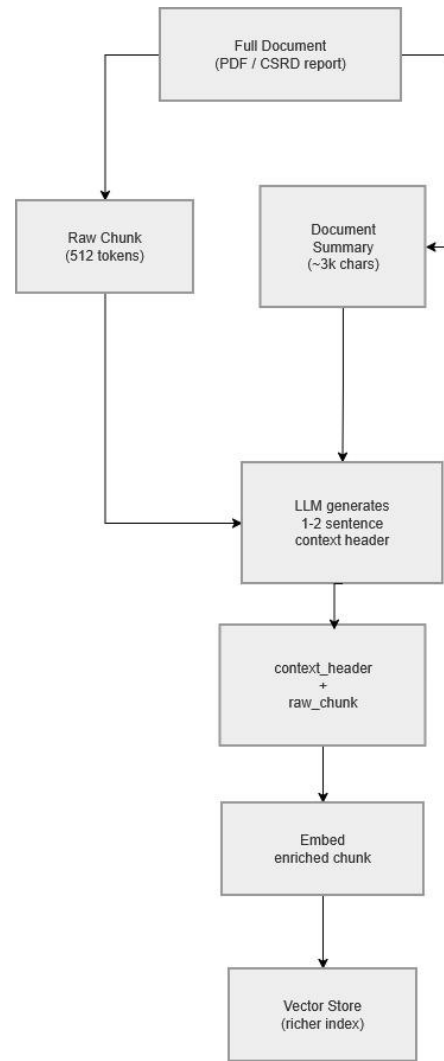
- Before storing each chunk, prepend a short LLM-generated context summary
- The embedding now carries document-level context → retrieval improves dramatically
- 💰 Cost consideration: adds N LLM calls at index time (one-time cost). Use prompt caching to reduce by ~80%. Negligible at query time.



Not the same as metadata filtering — this is semantic context, not structured tags

- Works best on long documents with implicit references (annual reports, CSRD filings, 10-K)

- Particularly powerful for ESG documents: **sustainability reports are dense, cross-referenced, and terminology-heavy**
- Naive chunking consistently loses the referential thread — contextual chunking fixes this
- Stack with BM25 hybrid retrieval for maximum recall on structured ESG data



MCP: Model Context Protocol — RAG Meets the Real World

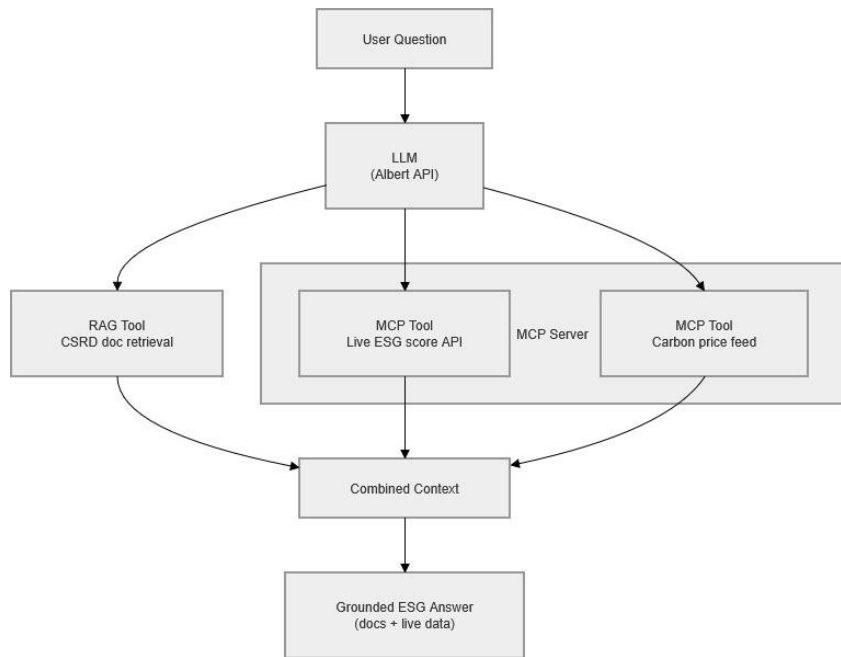
RAG is powerful — but it only knows what you've indexed. MCP solves the last mile: giving LLMs live, structured access to external systems.

🔌 What is MCP?

- Open standard (Anthropic, 2024) for connecting LLMs to external tools, APIs, and data sources
- MCP server = a service that exposes tools the LLM can call. MCP client = the LLM runtime.
- RAG: best for static or semi-static document corpora (ESG reports, regulatory filings)
- MCP + RAG: combine both — retrieve document context AND query live data in one pipeline

🔗 Resources: <https://modelcontextprotocol.io/docs/getting-started/intro>

- Industry adoption: **Cursor, Claude Desktop, Copilot, and major LLM platforms all support MCP natively in 2025**
- This is the direction enterprise AI is moving — static retrieval alone is not enough
- Knowing MCP makes you significantly more hireable in enterprise AI roles in 2025



Tool Calling: Giving LLMs Hands

An LLM without tools can only reason over what is in its context window. Tool calling lets it take actions: call APIs, query databases, run code, retrieve data in real time.



How Tool Calling Works

- You define tools (functions with JSON schemas). The LLM decides when to call them and with what arguments. Results are injected back into context.
- This is the mechanism behind RAG search tools, web search, code execution, and MCP servers.

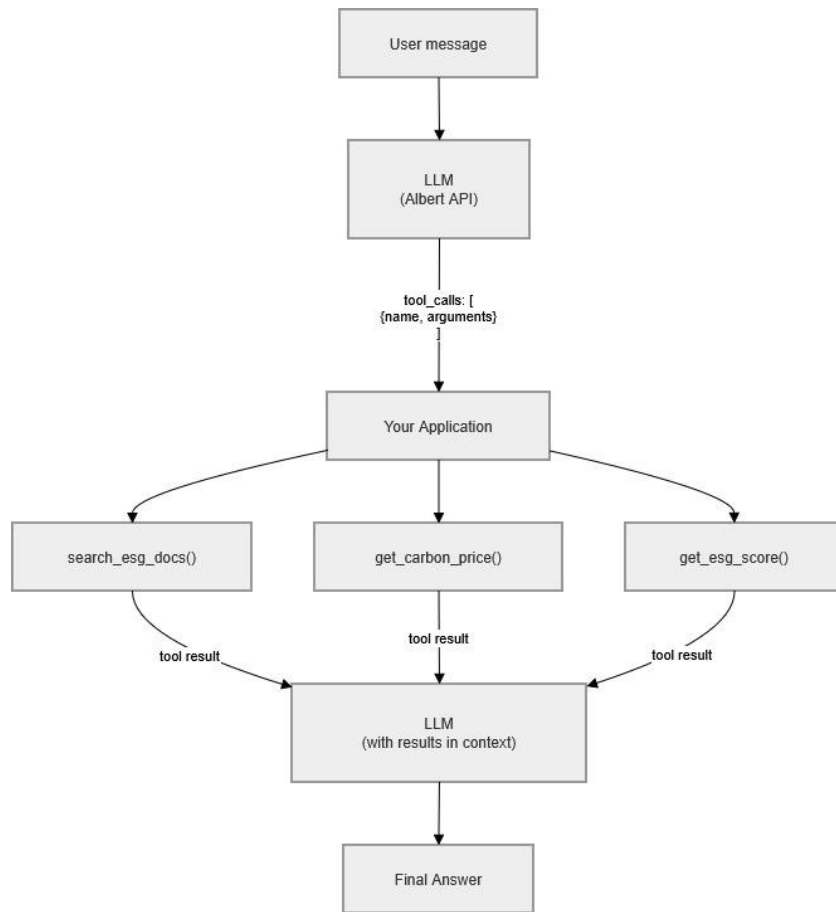


Tool Calling in Practice (OpenAI-compatible)

- Define a tool: { "name": "search_esg_docs", "description": "...", "parameters": { "query": "string" } }.
- Pass tools array to /v1/chat/completions. Model returns tool_calls with name + arguments. You execute the function and return results as tool role message.



The LLM does not execute code — it emits a structured JSON call. Your application executes it and feeds results back. This loop can repeat multiple times.



Agentic AI: From Single Calls to Multi-Step Reasoning

A standard LLM call is one-shot: prompt in, answer out. Agentic AI chains multiple reasoning steps, tool calls, and self-corrections to solve complex tasks autonomously.

💡 The Agent Loop

- Think → Act → Observe → Think → Act → Observe... until the agent decides the task is complete.
- Each iteration: reason → call tool → observe result → update plan.
- MCP/ToolCall fits here: the agent calls MCP servers as tools, enabling live ESG data, web search, code execution — all in the same loop.

⚙️ ReAct: the Dominant Agentic Pattern

- ReAct = Reason + Act.
- The LLM alternates between:
Thought: "I need to find TotalEnergies Scope 1 emissions..."
Action: `search_esg_docs("TotalEnergies Scope 1 2024")`
Observation: [retrieved chunk]
Thought: "Now I need to verify against their 2030 target..."
Final Answer: ...

⚠️ Agents are powerful but unpredictable. They can loop, hallucinate tool arguments, or call the wrong tool. Production agents need: `max_iterations` limit, tool output validation, human-in-the-loop for high-stakes decisions

- ESG agent example: given a CSRD compliance form, the agent autonomously identifies missing data points, queries the right report sections, cross-references targets, and fills in the form — flagging uncertainties for analyst review

